5G Standalone Architecture Implementation Using Open5GS and UERANSIM

Abstract

This report presents a detailed implementation of a 5G Standalone (SA) architecture using Open5GS and UERANSIM on Ubuntu. The work includes core deployment, gNB integration, UE registration, network slicing, Mobile Edge Computing (MEC), and an end-to-end smart healthcare use case.

Environment Setup

Operating System: Ubuntu

Open5GS installed from source at ~/open5gs

UERANSIM installed at ~/UERANSIM

MongoDB used as subscriber database backend

Experiment 5: Open5GS Core Setup

Objective: Deploy and validate Open5GS 5G Core.

Procedure:

- Install Open5GS and MongoDB

- Start services: sudo systemctl start open5gs-*

- Verify: systemctl status open5gs-*

- Start WebUI:

cd ~/open5gs/webui

npm install

npm run dev

- Access: http://localhost:9999

- Add subscriber (IMSI: 999700000000001, APN: internet)

Result: Core services operational, subscriber successfully added.

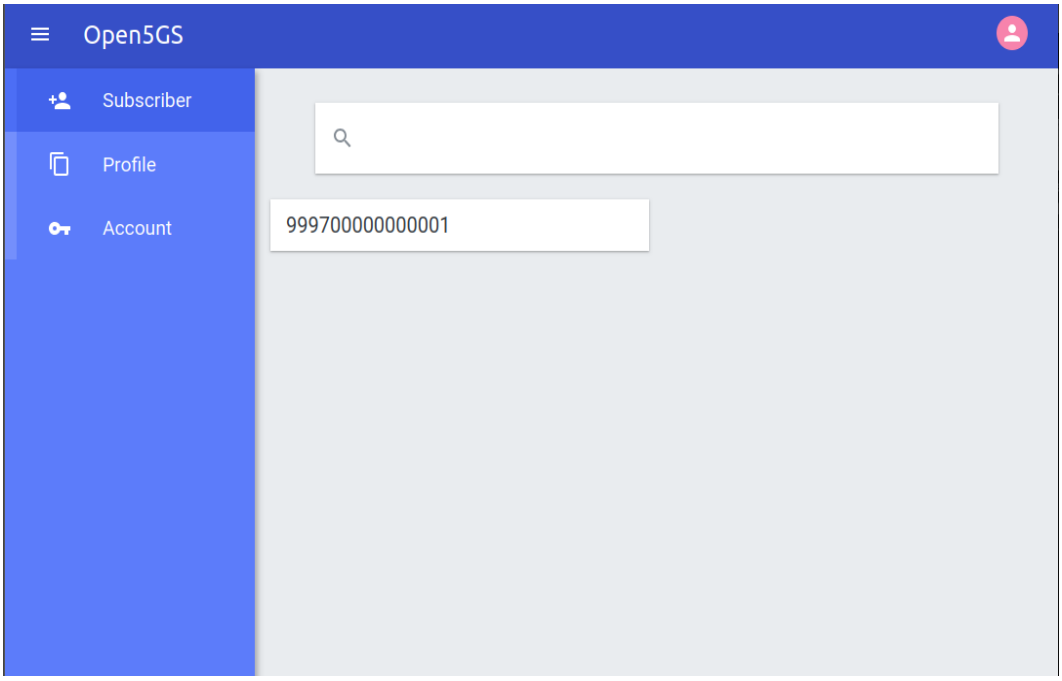


Figure 1: Open5GS WebUI Access

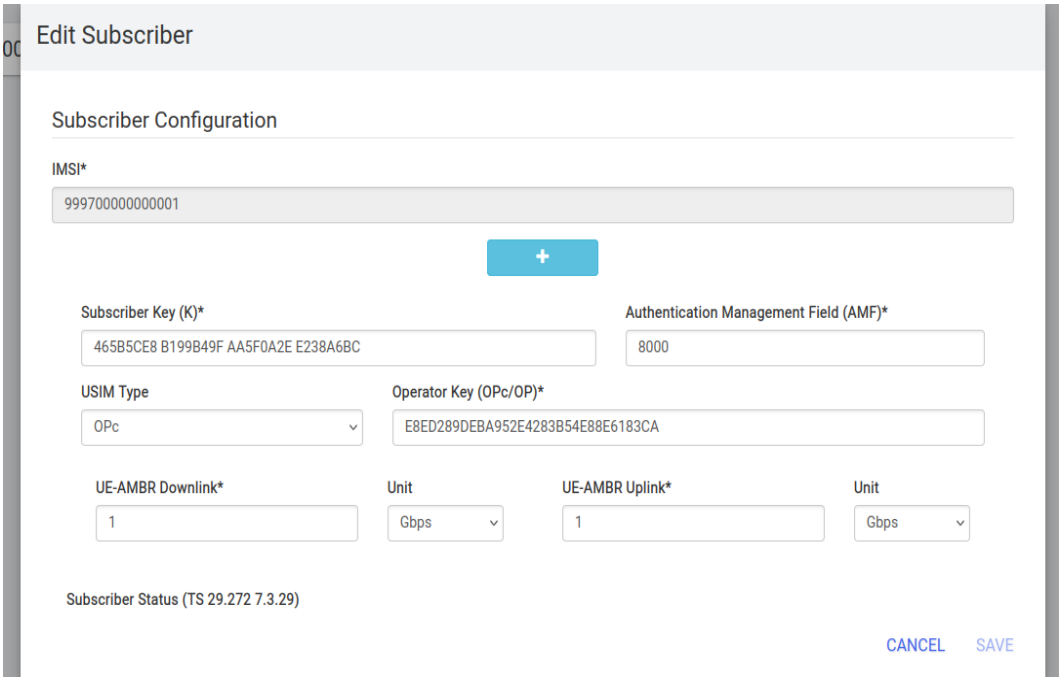


Figure 2: Subscriber Configuration

Experiment 6: gNB Integration

Objective: Connect simulated gNB to 5G Core.

Procedure:

- Configure gNB: ~/UERANSIM/config/open5gs-gnb.yaml

- Ensure AMF IP matches Open5GS

- Start gNB:

sudo ./build/nr-gnb -c config/open5gs-gnb.yaml

Result: NG Setup successful and gNB connected to AMF.

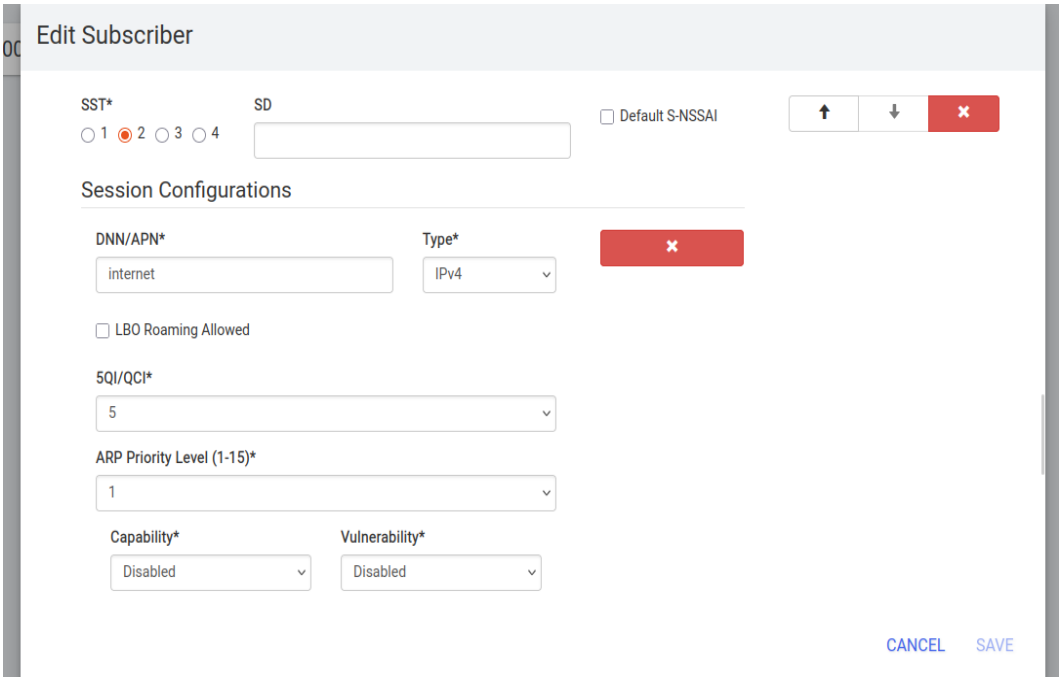


Figure 3: NG Setup Successful

Experiment 7: UE Registration and PDU Session

Objective: Register UE and establish data session.

Procedure:

- Configure UE: ~/UERANSIM/config/open5gs-ue.yaml

- Use IMSI, Key, OPC from subscriber

- Start UE:

sudo ./build/nr-ue -c config/open5gs-ue.yaml

- Fix SQN mismatch via resync

Commands (optional NAT):

sysctl -w net.ipv4.ip_forward=1

iptables -t nat -A POSTROUTING -o eth0 -j MASQUERADE

Result: UE registered successfully, PDU session established, IP assigned (10.45.0.2).

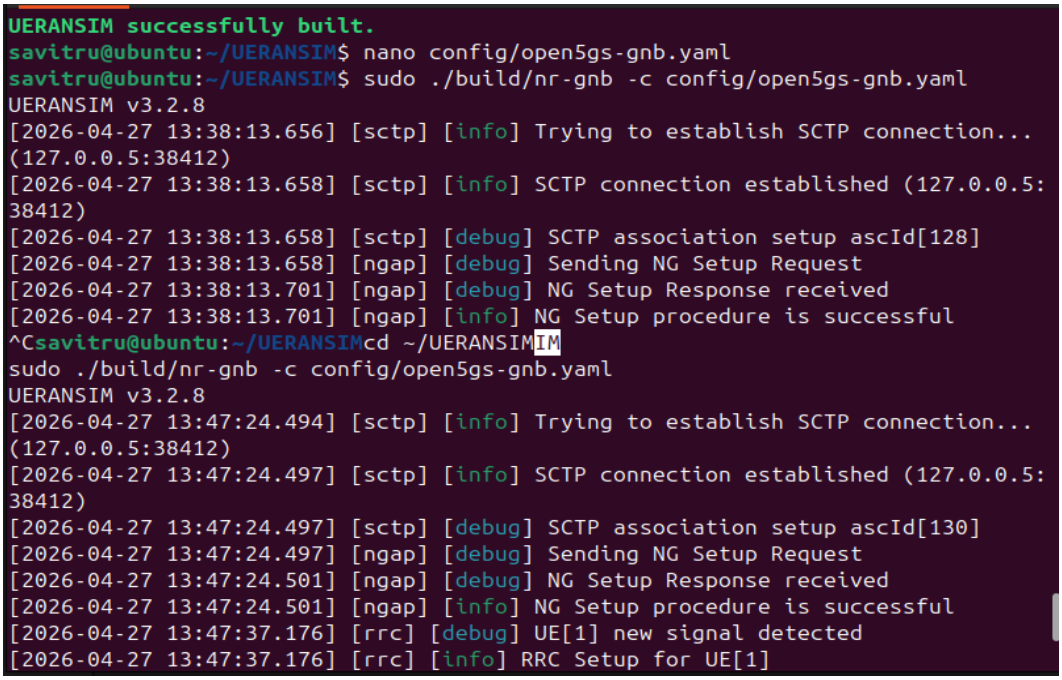


Figure 4: UE Registration Accept

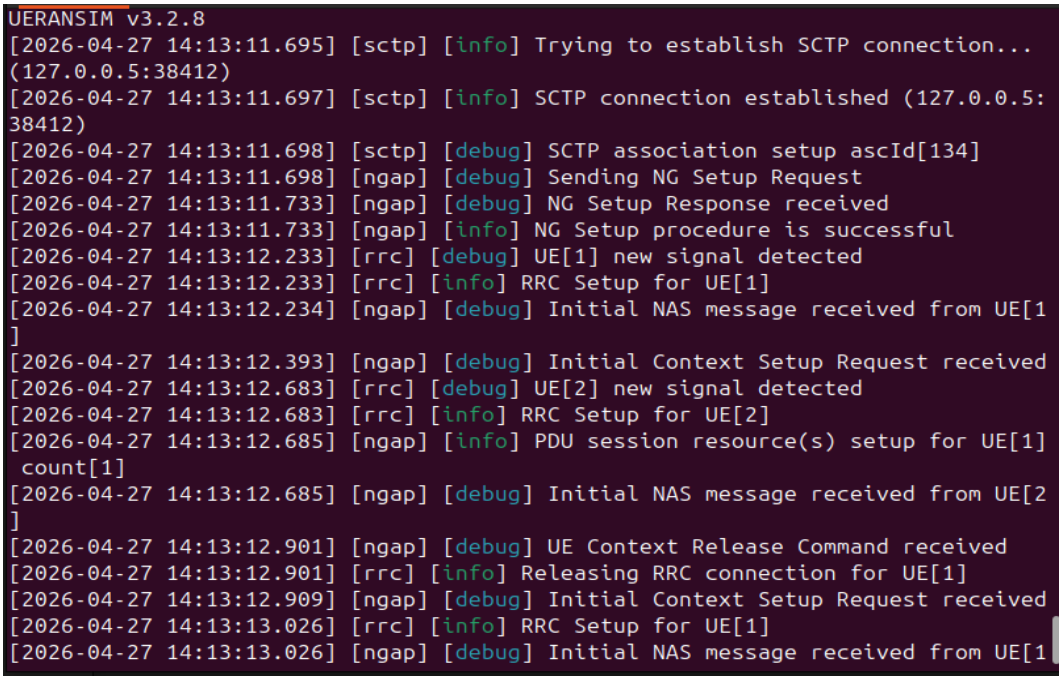


Figure 5: PDU Session Established

Experiment 8: Network Slicing

Objective: Implement multiple network slices.

Procedure:

- Modify AMF config: /etc/open5gs/amf.yaml

- Add S-NSSAI (SST 1 and SST 2)

- Restart AMF:

sudo systemctl restart open5gs-amfd

- Update subscriber slices in WebUI

- Test UE with sst:1 and sst:2

Result: UE successfully registers across multiple slices.

Figure 6 (Textual): AMF configured with multiple S-NSSAI enabling slice-based registration.

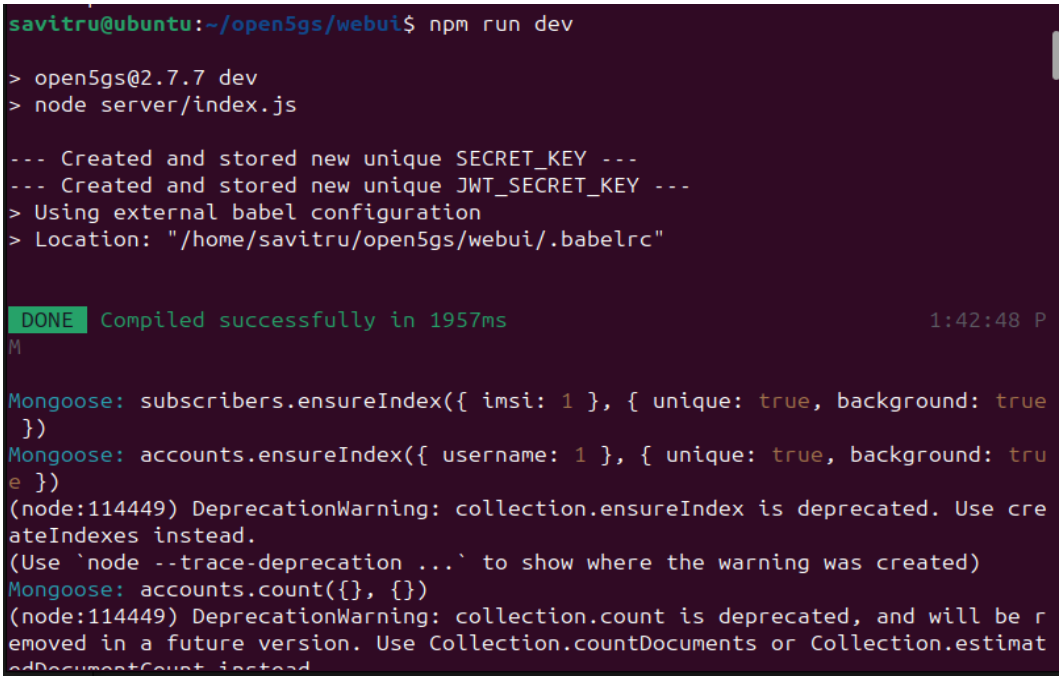


Figure 7: UE Slice Selection

Experiment 9: Mobile Edge Computing

Objective: Evaluate latency difference between edge and cloud.

Procedure:

- Start local server:

python3 -m http.server 8080

- Measure latency:

ping 127.0.0.1 -c 5

ping 8.8.8.8 -c 5

Result: Edge latency (~0.03 ms) significantly lower than cloud (~120 ms).

Experiment 10: End-to-End Smart Healthcare Use Case

Scenario: URLLC healthcare monitoring using MEC.

Architecture: Open5GS core + UERANSIM gNB + UE + edge server

Execution:

- UE configured for SST 2 (URLLC)

- Edge server running

- Latency comparison performed

Result: Demonstrates low-latency communication suitable for healthcare applications.

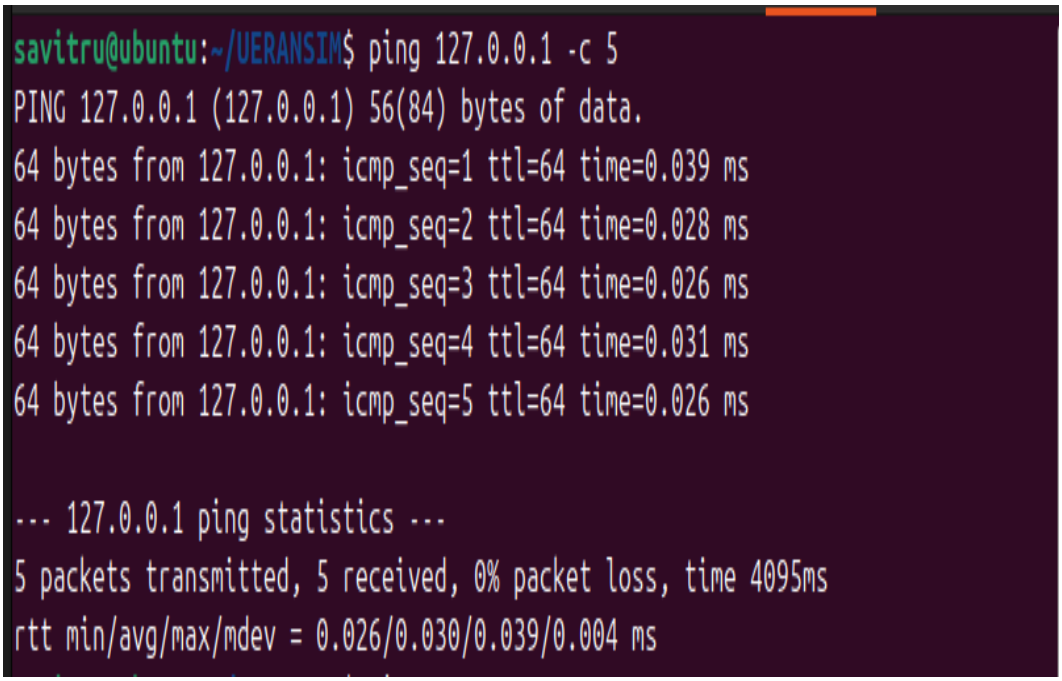


Figure 9: End-to-End Demonstration

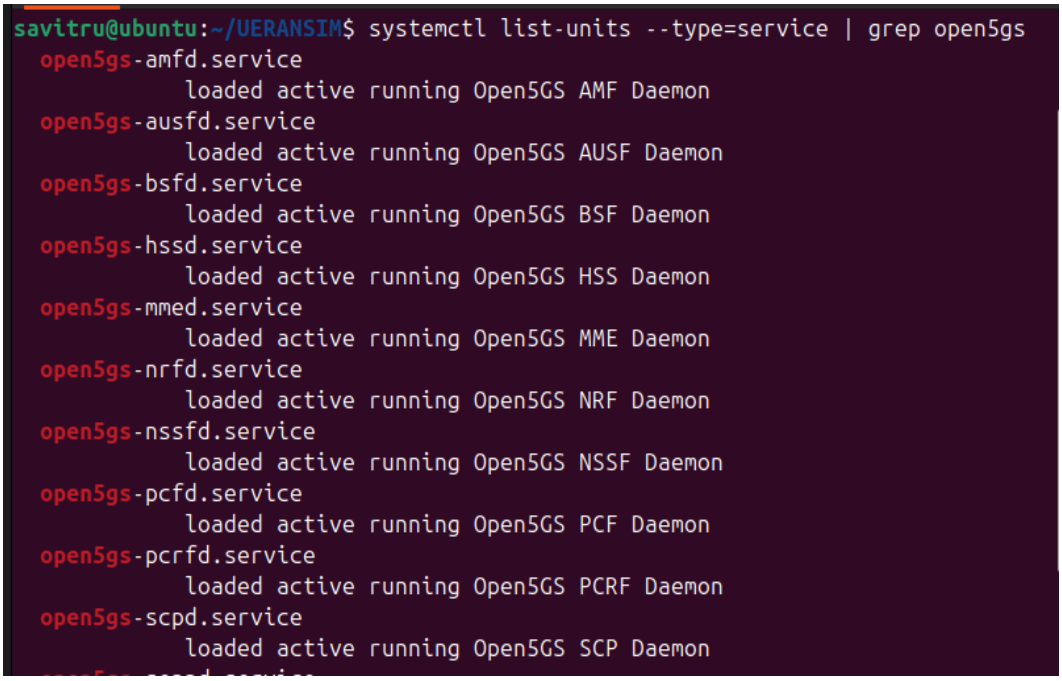


Figure 10: Activated Running Services

Conclusion

The implementation successfully demonstrates a complete 5G SA architecture including core deployment, RAN integration, UE connectivity, slicing, and MEC. The system validates real-world low-latency applications such as smart healthcare.